KTON
Audit Report

Mon May 26 2025

contact@bitslab.xyz https://twitter.com/tonbit_

TonBit

KTON Audit Report

1 Executive Summary

1.1 Project Information

Description	Liquid Staking (LST) is a protocol that connects TON holders of all caliber with hardware node operators to participate in TON Blockchain validation through assets pooling
Type	DeFi
Auditors	TonBit
Timeline	Thu Apr 03 2025 - Mon May 26 2025
Languages	FunC
Platform	Ton
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/KTON-IO/liquid-staking-contract
Commits	b0352cdce27f326644f8b3ad3411eed75812c614 10786c786a01ccebffbaa1ad105e536b0511fdd9 b0a69b51889fd6858c57816c6baf408aecd10401 01be54de555f5591aa2fa186c024f4c17dcb4872

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
CCO	elector/config-code.fc	86b5937b60b948d8aae93095bfba876136759c83
ECO	elector/elector-code.fc	2b05a7eedcd1d37452028076d7035a2463aacb6d
NCU	contracts/network_config_utils.func	5bbd9279574035906099792bb1a2f6003cfb963a
PMH	contracts/pool_mint_helpers.func	5b4e94143afcbc54348506bdf048ab34ce2fcde8
VER	contracts/versioning.func	b53c2212dda2dfe490acfad0b1c95e38558a325d
ASS	contracts/asserts.func	183d8096a46b11532a49ba388be17ea146c05ddd
NCO	contracts/payout_nft/nft-collection.func	e90656be3eb26afdba3799d555a3ee4f4f892a37
TYP	contracts/payout_nft/types.func	b842d47f8664697a9259645bbb50eb91ee0d3d98
MUT	contracts/payout_nft/metadata_utils.func	63a7a89fd8d860d2fee1b6d277d01555f5ffac78
STD	contracts/payout_nft/stdlib.func	7a308e61e2a9017c8a923d0cad96d41db0d76f69

PAR	contracts/payout_nft/params.func	0f9f4a2a31d1398374b6a8a2cb841dec265ba7ec
OCO	contracts/payout_nft/op-codes.func	764c3348a51196578cf99c172e39005d47b09d14
MES	contracts/payout_nft/messages.func	a0c095360e5c2ad16b6e5fd2184f9595b68d4ab1
ERR	contracts/payout_nft/errors.func	3a2a8b71e2b134ca355b393be7e585316cce82fa
NIT	contracts/payout_nft/nft-item.func	c9b8fc9c714c8bafca6f5d8adf355c03fa5cff49
TYP1	contracts/types.func	dd0249b9dcaaab159abea843497d1e6dd9407885
MUT1	contracts/metadata_utils.func	9fb25672739d7e2cf6da2cbd578d364a6606da42
PST	contracts/pool_storage.func	880c2ac81679de5863b9f2bd3c25acff288b03c9
OCO1	contracts/op-codes.func	f4632bead38e628905c4e82cf9155071bed2ae7d
LIB	contracts/librarian.func	04017e4d2000102de80ad00a866ce6580b40bf34
DPA	contracts/dao_params.func	a38462fc812128e50bf3c786cbd24b6636eb0bd6
CON	contracts/controller.func	0d332172d816a549ffc2cac800e17f143121acef

MES1	contracts/messages.func	afc63199ac393dd01be37c9f8c499a3e4ab72de2
ACA	contracts/address_calculations.func	acac2cc54b0f3288d60ad8a794930b17fa9ff1e1
RHE	contracts/roles_helper.func	6279b3fd604b02a654438e62f68ee2b531032471
SRE	contracts/sudoer_requests.func	3930b608d675da7a7fa087e8c5f1617d82891a55
POO	contracts/pool.func	c2d2aabf8c98d5cdeb09b25bc7af0a586a93e197
ERR1	contracts/errors.func	105471da5d31a57e12e10e25f88644315183b735

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	5	5	0
Informational	2	2	0
Minor	0	0	0
Medium	3	3	0
Major	0	0	0
Critical	0	0	0

1.4 TonBit Audit Breakdown

TonBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by KTON to identify any potential issues and vulnerabilities in the source code of the KTON smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 5 issues of varying severity, listed below.

ID	Title	Severity	Status
CON-1	Missing Fee Check in Balance Validation	Medium	Fixed
CON-2	Incorrect Comment	Informational	Fixed
POO-1	Incorrect Rounding Direction	Medium	Fixed
POO-2	Incorrect Range Setting	Medium	Fixed
POO-3	Incomplete Time Check	Informational	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the KTON Smart Contract :

Sudoer

- `upgrade` : Upgrade the contract

Governor

- `set_roles` : Set the address of each management role of the contract
- `set_deposit_settings` : Configure deposit/withdrawal status and withdrawal fee ratio
- `set_governance_fee` : Set the protocol share ratio
- `unhalt` : Unsuspend the contract
- `prepare_governance_migration` : Start the governance rights migration process
- `set_sudoer` : Set the address of the super administrator
- `return_available_funds` : Mandatory return of available funds to the pool

Interest_manager

- `set_interest` : Dynamic adjustment of the base lending rate
- `set_operational_params` : Set loan operation parameters

Halter

- `halt` **/** `partial_halt` : Manage contract state

User

- `deposit` : Deposit TON to the protocol for liquidity tokens
- `withdraw` : Destroy pool tokens to redeem TON
- `donate` : Donate TON to the pool free of charge

Validator

- `deploy_controller` : Deploy dedicated controller contracts for validation nodes
- `new_stake` : Initiate a new stake

- `recover_stake` : Apply to unstake
- `send_request_loan` : Apply for a loan from the pool
- `return_unused_loan` : Return unused loans
- `withdraw_validator` : Withdraw the Validator's own funds
- `update_validator_hash` : Update the validation node collection hash

Approver

- `approve` **/** `approve_extended` **/** `disapprove` : Set the loan related parameters

4 Findings

CON-1 Missing Fee Check in Balance Validation

Severity: Medium

Status: Fixed

Code Location:

contracts/controller.func#479-506

Descriptions:

The `controller` contract lacks proper fee-related checks when validating balances. For example, in the `op == controller::return_unused_loan` message handler, the contract verifies that `balance >= MIN_TONS_FOR_STORAGE + borrowed_amount`. However, it does not account for gas fees or fwd fees, which may result in the contract's actual remaining balance falling below `MIN_TONS_FOR_STORAGE + borrowed_amount` after the message is sent.

```
            if(balance >= MIN_TONS_FOR_STORAGE + borrowed_amount) {
              send_msg(pool,
                    borrowed_amount, ;; TODO add fee???
                    begin_cell().store_body_header(pool::loan_repayment,
query_id).end_cell(),
                    msgflag::BOUNCEABLE,
                    sendmode::PAY_FEES_SEPARATELY);
```

Suggestion:

It is recommended to enhance the balance validation logic by including checks for gas and forwarding fees.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CON-2 Incorrect Comment

Severity: Informational

Status: Fixed

Code Location:

contracts/controller.func#153-157

Descriptions:

In the `op == elector::recover_stake_ok` message, when calculating `amount_to_return`, the comment is currently:

```
;; return max of borrowed_amount or borrowed_amount + profit_share * profit
```

However, this is inaccurate. The `borrowed_amount` here actually represents the loan principal, so the comment should reflect that.

Suggestion:

Update the comment to:

```
;; return max of borrowed_amount or principal + profit_share * profit
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-1 Incorrect Rounding Direction

Severity: Medium

Status: Fixed

Code Location:

contracts/pool.func#410-416

Descriptions:

In the `pool` contract, under the `op == pool::request_loan` message, the contract calculates the maximum loanable amount as follows:

```
int creditable_funds = balance
                - muldiv_extra(requested_for_withdrawal, total_balance, supply)
                - MIN_TONS_FOR_STORAGE;
var [_, _, _, borrowed, _, _, _] = current_round_borrowers;
int available_funds = min( creditable_funds,
                  muldiv((DISBALANCE_TOLERANCE_BASIS / 2) + disbalance_tolerance,
total_balance, DISBALANCE_TOLERANCE_BASIS)
                  - borrowed );
```

The rounding direction for `creditable_funds` should always favor the protocol—specifically, it should be rounded down. However, the function `muldiv_extra(requested_for_withdrawal, total_balance, supply)` is already rounding down, which causes `creditable_funds` to be slightly overestimated due to rounding error.

Suggestion:

It is recommended to explicitly round down the result of `creditable_funds` to ensure conservative and protocol-safe behavior.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-2 Incorrect Range Setting

Severity: Medium

Status: Fixed

Code Location:

contracts/pool.func#388-450

Descriptions:

In the `op == pool::request_loan` message,

```
int total_loan = current_round_borrowers~add_loan(sender_address, actual_loan, interest);
throw_unless(error::total_credit_too_high, total_loan <= max_loan_per_validator + interest);
```

Here `total_loan` returns the principal of the loan, but here the comparison is indeed the maximum personal loan plus interest.

Suggestion:

It is recommended that modified to `throw_unless(error::total_credit_too_high, total_loan <= max_loan_per_validator);` .

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-3 Incomplete Time Check

Severity: Informational

Status: Fixed

Code Location:

contracts/pool.func#388-396

Descriptions:

In the `op == pool::request_loan` request, only

```
throw_unless(error::too_early_borrowing_request, now() > utime_until -
elections_end_before - credit_start_prior_elections_end);
```

was checked during the time check, and the complete time check was missing.

Suggestion:

Refer to the controller contract and add complete time verification

```
throw_unless(error::too_early_loan_request, now() > utime_until -
elections_start_before); ;; elections started
throw_unless(error::too_early_loan_request, now() > utime_until - elections_end_before -
allowed_borrow_start_prior_elections_end); ;; allowed to borrow
throw_unless(error::too_late_loan_request, now() < utime_until - elections_end_before); ;;
elections not yet closed
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.